

Tourism Management System

PROJECT REPORT

For

SEC 160 – FULL STACK DEVELOPMENT-1

Submitted by

Madhu Swapnika.G	AP23110011011
B.Harshavardhan Reddy	AP23110011305
Ch.Chennapa Naidu	AP23110011401
T.Ram Charan	AP23110011345

SEM: V

SECTION: J

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

of

SCHOOL OF ENGINEERING AND SCIENCES



SRM University-AP, Neerukonda, Andhra Pradesh 522240

December 2025

TABLE OF CONTENTS

- 1. Introduction**
- 2. Scenario-Based Introduction**
- 3. Target Audience**
- 4. Project Goals and Objectives**
- 5. Technologies Used**
 - 5.1 Frontend Technologies
 - 5.2 Backend Technologies
 - 5.3 Tools & Development Environment
- 6. Project Structure**
- 7. Project Documentation**
- 8. Application Modules and Features**
 - 8.1 Reusable Components Module
 - 8.2 Pages Module(output screenshots)
 - 8.3 Admin Module
 - 8.4 Services & API Module
- 9. Future Enhancements**
- 10.Key Features of the Application**
- 11.Conclusion**

Introduction

The Travel Booking Web Application is a modern, fully responsive platform designed to simplify the experience of exploring destinations, viewing travel packages, and managing bookings online.

Built using **React.js**, the application prioritizes performance, usability, and scalability through a component-based architecture.

This project focuses on delivering a seamless user journey with intuitive navigation, well-organized layouts, and fast-loading UI components. The backend API is integrated using Axios, ensuring smooth communication between the frontend interface and server data.

The system also includes an **admin module** designed for managing bookings, offering essential CRUD functionalities. Route protection ensures that only verified administrators can access restricted areas, maintaining system security and data integrity.

Scenario-Based Introduction

Imagine a user planning a vacation. They open the Travel Booking Web Application and instantly view a beautifully structured Home Page with featured destinations and travel packages.

The user explores the *Destinations* section, scrolling through curated cards showing various places, each with images, pricing, and brief descriptions.

Curious about a particular place, the user selects it, navigating to the *Destination Details* page. Here, they see detailed information, including itinerary, activities, and package highlights.

From there, the user browses travel packages and selects a package that suits their budget and preferences. The platform prompts them to log in or register to proceed with the booking.

Upon successful login, the user books the package with a single click, and the booking details are saved securely.

At the same time, an admin logs into the secure admin dashboard, where they can view all bookings, edit any incorrect details, or delete bookings that are no longer valid. The admin module ensures smooth backend operations, contributing to the overall management of the system.

Every part of this application is designed to deliver clarity, accessibility, and convenience to both users and admins.

Target Audience

The application primarily targets the following groups:

- **General users**

People who want a simple and intuitive way to browse destinations and book travel packages online.

- **Travel agencies / administrative staff**

Administrators who need easy access to booking records and need tools for editing, updating, or managing user bookings efficiently.

- **Students & developers**

Learners exploring how modern frontends interact with backend APIs using React.

Project Goals and Objectives

Main Goal

To build an accessible, efficient, and streamlined platform for users to browse travel options and place bookings while providing administrators with the tools needed for backend management.

Objectives

- Develop a **fully responsive and visually appealing** UI using Tailwind CSS
- Ensure smooth navigation using **React Router**
- Enable users to view destinations and packages through clean structured pages
- Implement **authentication** for login and registration
- Integrate backend APIs for dynamic content retrieval
- Provide admin-only features, including **secure CRUD operations**
- Apply reusable components to maintain consistency and reduce code duplication
- Maintain a scalable and easily extendable architecture

Technologies Used

Frontend Technologies

React.js

Used to build the entire user interface with a component-based approach that improves maintainability and reusability.

React Router

Handles client-side navigation without reloading pages, enabling a smooth user experience.

Tailwind CSS

Provides utility-first styling to ensure responsive, modern, and customizable UI components.

Axios

Used for making HTTP requests to communicate with backend APIs.

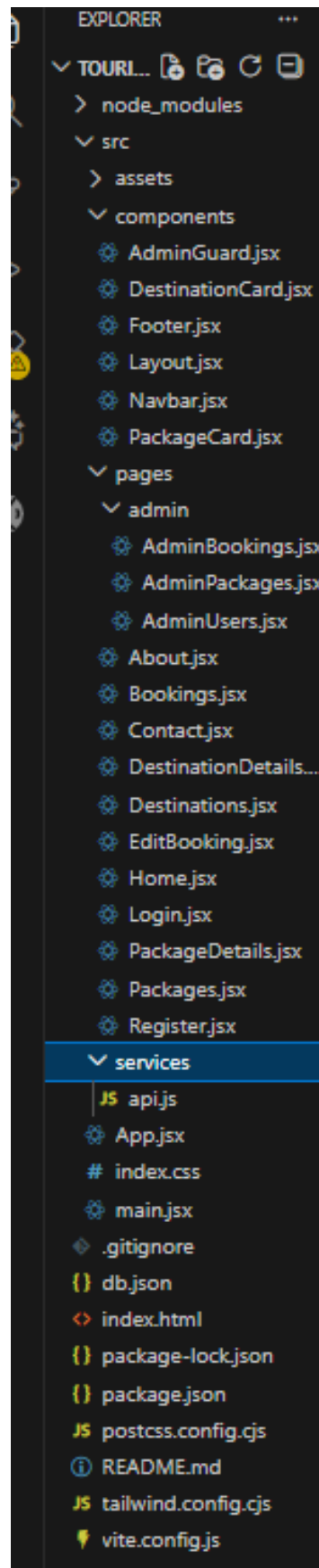
Backend Technology

- REST API for handling authentication, destinations, packages, and booking endpoints.
- Token-based authentication to secure admin routes.

Tools

- **VS Code** – Code editor
- **Git / GitHub** – Version control and collaboration
- **Postman** – API testing and debugging

Project Structure:



System Architecture Overview

The system consists of the following major segments:

User Interface Layer

The UI layer interacts directly with the user and displays dynamic data. Key responsibilities include:

- Rendering destination and package lists
- Displaying individual details pages
- Handling login/register and user interaction
- Navigating using a clean, structured layout

Application Logic Layer

This layer processes all user actions such as:

- Form validation
- Conditional rendering
- Handling API results

Admin Module

The admin module provides:

- Restricted access to authorized users
- Tools for managing bookings
- CRUD functionality (Create, Read, Update, Delete)
- Protected pages using AdminGuard

Service Layer

A dedicated services folder contains:

- Axios configuration
- API request functions
- Token handling
- Error management

This provides a centralized communication channel between frontend and backend.

Application Modules and Features

Reusable Components Module

All UI components in the system follow the React philosophy of reusability. This ensures a consistent look and feel across the application.

Components include:

- **Navbar.jsx** — Displays global navigation links
- **Footer.jsx** — Shown on every page for consistency
- **Layout.jsx** — Wraps page content with a common structure
- **DestinationCard.jsx & PackageCard.jsx** — Render dynamic content as reusable cards
- **AdminGuard.jsx** — Protects admin routes from unauthorized users

These components reduce code repetition and allow effortless expansion of the UI in the future.

Admin Module

The Admin Module is a core part of the project:

- **Bookings.jsx** displays all user bookings
- **EditBooking.jsx** allows modification of booking details
- CRUD operations make it easy to manage records
- Admin routes are protected by **AdminGuard.jsx**

This ensures complete control over the backend data from the admin interface.

Services Module

Located in **services/api.js**, this module manages:

- API calls using Axios
- Token-based authentication
- Error handling (e.g., expired tokens, failed requests)

Project Documentation:

Milestone 1: Project Setup and Configuration

1. Installation of Required Tools and Software

To begin the project, the required tools and libraries are installed. The project is built using:

- React.js — For building the user interface.
- React Icons — For adding icons inside components.
- Material-UI (MUI) — For UI styling and layout.
- react-codemirror2 — For the interactive code editor functionality.

After creating/opening the project folder, the dependencies are installed.

Reference links used during setup:

- React Installation Guide: <https://react.dev/learn/installation>
- UI Component Setup Reference:
<https://react-bootstrap-v4.netlify.app/getting-started/introduction/>

Milestone 2: Web Development

1. Setting up the React Application

The React application is created and configured to support routing and UI components.

App.jsx Component Overview

- ./App.css — Provides styling for the App component.
- Home (./components/Home) — Main UI component of the application.
- DataProvider (./context/DataProvider) — Provides global state using React Context API.

The `<DataProvider>` wraps the `<Home/>` component so the entire application can access shared state values like HTML, CSS, and JavaScript code.

Finally, App is exported as the default component.

```

src > App.jsx > ...
1 import React from 'react'
2 import { Routes, Route } from 'react-router-dom'
3 import Home from './pages/Home'
4 import Destinations from './pages/Destinations'
5 import DestinationDetails from './pages/DestinationDetails'
6 import Packages from './pages/Packages'
7 import PackageDetails from './pages/PackageDetails'
8 import Bookings from './pages/Bookings'
9 import EditBooking from './pages/EditBooking'
10 import About from './pages/About'
11 import Contact from './pages/Contact'
12 import Login from './pages/Login'
13 import Register from './pages/Register'
14 import Layout from './components/Layout'
15 import AdminUsers from './pages/admin/AdminUsers'
16 import AdminBookings from './pages/admin/AdminBookings'
17 import AdminPackages from './pages/admin/AdminPackages'
18 import AdminGuard from './components/AdminGuard'
19
20 export default function App(){
21   return (
22     <div className="min-h-screen flex flex-col bg-black">
23       <Routes>
24         <Route path="/login" element={<Login />} />
25         <Route path="/register" element={<Register />} />
26         <Route path="/" element={<Layout><Home/></Layout>} />
27         <Route path="/destinations" element={<Layout><Destinations/></Layout>} />
28         <Route path="/destination/:id" element={<Layout><DestinationDetails/></Layout>} />
29         <Route path="/packages" element={<Layout><Packages/></Layout>} />
30         <Route path="/packages/:id" element={<Layout><PackageDetails/></Layout>} />
31         <Route path="/bookings" element={<Layout><Bookings/></Layout>} />
32         <Route path="/bookings/edit/:id" element={<Layout><EditBooking/></Layout>} />
33         <Route path="/about" element={<Layout><About/></Layout>} />
34         <Route path="/admin/users" element={<Layout><AdminGuard><AdminUsers/></AdminGuard></Layout>} />
35         <Route path="/admin/bookings" element={<Layout><AdminGuard><AdminBookings/></AdminGuard></Layout>} />
36         <Route path="/admin/packages" element={<Layout><AdminGuard><AdminPackages/></AdminGuard></Layout>} />
37         /* How-it-works merged into About - route removed */
38         <Route path="/contact" element={<Layout><Contact/></Layout>} />
39       </Routes>
40     </div>
41   )
42 }
43

```

Description:

This is the main routing file (App.jsx) of the Tourist Management System.

It imports all pages, admin components, layout components, and authentication guards.

Using **React Router v6**, different routes are defined for Login, Register, Home, Destinations, Packages, Bookings, About, Contact, and Admin pages.

Each route uses the `<Route>` component, and most pages are wrapped inside the `<Layout>` component so that the navbar and footer appear consistently.

Admin routes are protected using `AdminGuard`, ensuring only authenticated admin users can access those pages.

2. Designing UI Components

During this phase:

- All required React components are created.
- Layout and styling are implemented using **Material-UI** and custom CSS.
- Navigation structure is defined as per the application flow.

```
src > # index.css > ...
1  @tailwind base;
2  @tailwind components;
3  @tailwind utilities;
4
5  /* small helper utilities */
6  .container { max-width: 1100px; }
7
8  /* Custom animations */
9  @keyframes fade-in {
10   from {
11     opacity: 0;
12     transform: translateY(30px);
13   }
14   to {
15     opacity: 1;
16     transform: translateY(0);
17   }
18 }
19
20 @keyframes slide-in-up {
21   from {
22     opacity: 0;
23     transform: translateY(40px);
24   }
25   to {
26     opacity: 1;
27     transform: translateY(0);
28   }
29 }
30
31 @keyframes pulse-glow {
32   0%, 100% {
33     box-shadow: 0 0 20px rgba(59, 130, 246, 0.5);
34   }
35   50% {
36     box-shadow: 0 0 40px rgba(59, 130, 246, 0.8);
37   }
38 }
39
40 .animate-fade-in {
41   animation: fade-in 0.8s ease-out;
42 }
43
44 .animate-slide-in-up {
45   animation: slide-in-up 0.8s ease-out;
```

Description:

This is the root HTML file of the project used by Vite.

It contains the basic HTML document structure with a `<div id="root"></div>` where the entire React application is rendered.

The `<script type="module" src="/src/main.jsx"></script>` line loads the React entry file, which initializes the application.

The <meta> tags define character encoding and enable responsive layout for all screen sizes.

3. Implementing Frontend Logic

Frontend logic includes:

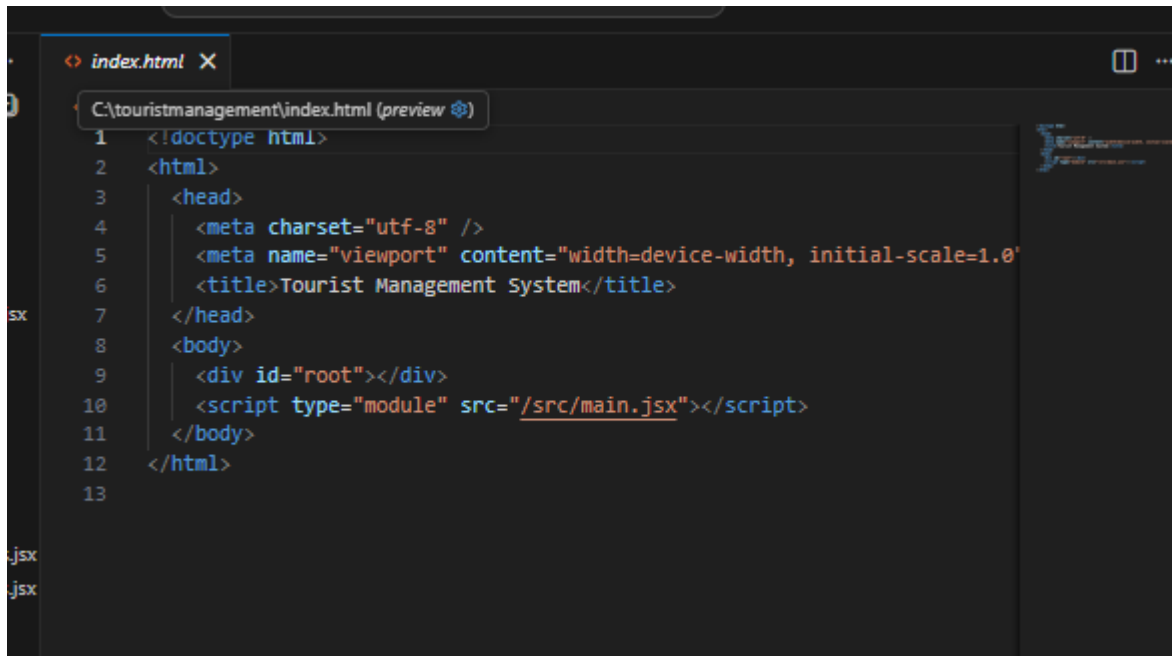
- Integration of UI components with context state.
- API integration (if used in future expansion).
- Data binding with CodeMirror editors through React state and context.

DataProvider Component

Code Description

- A **DataContext** is created using `createContext(null)`.
- The **DataProvider** component stores three state variables:
 - `html`
 - `css`
 - `js`
- These are managed using `useState` and shared with all nested components.
- `DataContext.Provider` returns these values and setter functions.
- Any child component can read or update the values using `useContext(DataContext)`.

This enables synchronized editing and output generation for HTML, CSS, and JavaScript code throughout the app.



```
1 <!doctype html>
2 <html>
3   <head>
4     <meta charset="utf-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Tourist Management System</title>
7   </head>
8   <body>
9     <div id="root"></div>
10    <script type="module" src="/src/main.jsx"></script>
11  </body>
12 </html>
13
```

Description:

This preview shows the same index.html file as it appears inside the editor.

It confirms that the React app will mount inside the `<div id="root">`, and Vite will compile and serve the project using the entry file `main.jsx`.

This file is essential because it acts as the **single HTML page** for the entire React application (SPA — Single Page Application).

Project Demo link:

https://drive.google.com/file/d/13TDDEYMtn95JDGzVQTqvvZ6Xr3ZJqnRH/view?usp=drive_link

Code explanation video :

https://drive.google.com/file/d/1sWO4SqjVSQD31Y_tWB-OZkUI4_YIScot/view?usp=drive_link

Code-Editor Component

Code Description

- The component uses `useState` to maintain editor open/close toggle.
- Imports:
 - `CodeMirror` styles and modes
 - `ControlledEditor` from `react-codemirror2`
 - Material-UI components (`Box`, `Typography`, `Button`, etc.)
 - Icons for editor heading and fullscreen toggle

Styled components include:

1. **Container** — Manages layout and spacing.
2. **Header** — Displays the editor title.
3. **Heading** — Shows the icon, title, and color styling.

The editor functionality includes:

- Syntax highlighting
- Linting
- Line numbers
- Theme setup

The `handleChange` function updates the code and passes it back to the parent using `onChange`.

Finally, the `Editor` component is exported for use inside the main `Code` component.

Project Execution

After writing all components, the application is run using:

`npm start`

Or if using Vite:

`npm run dev`

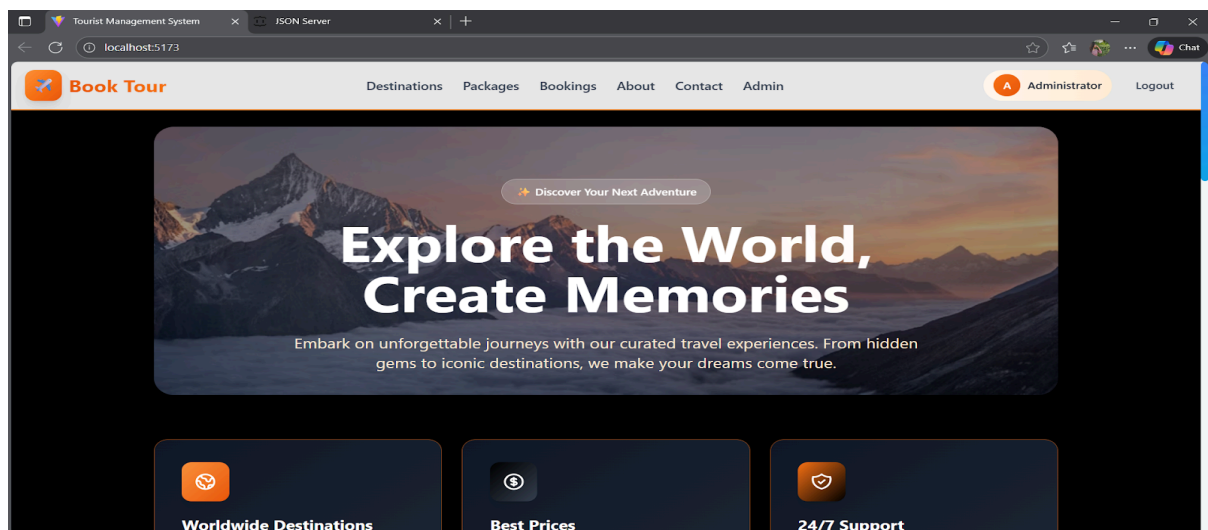
This starts the development server and launches the application.

Pages Module (Output screenshots):

This module contains all the user-facing screens:

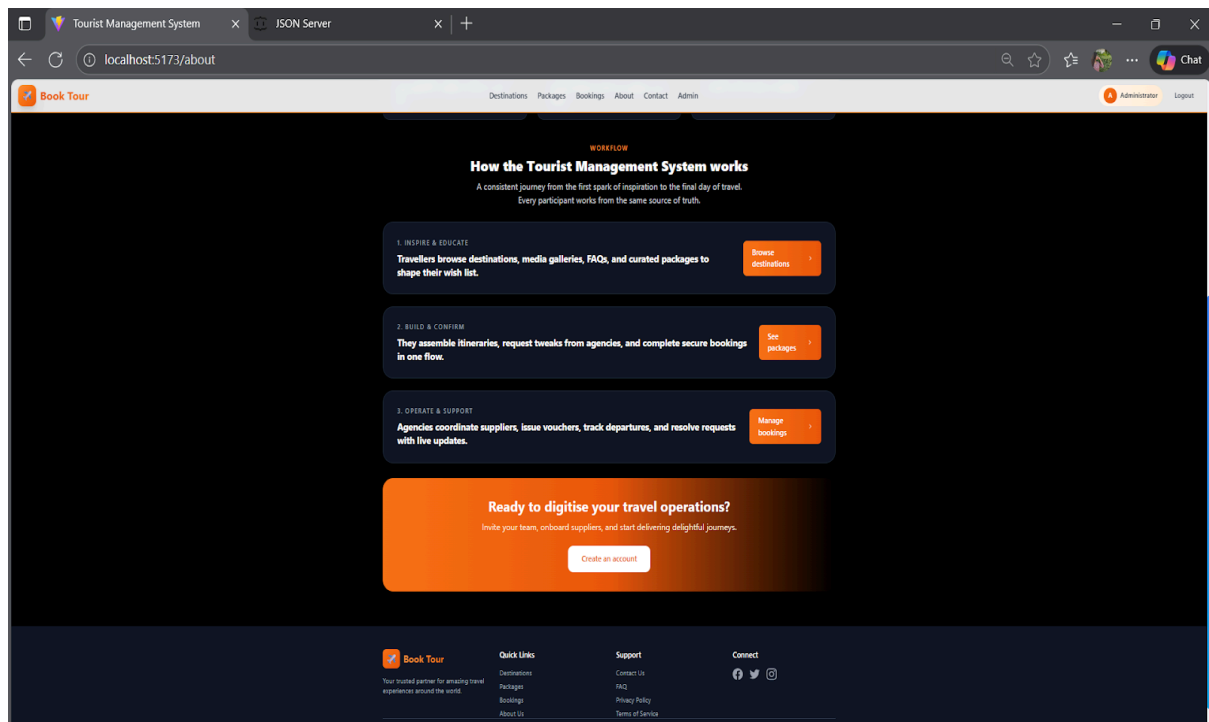
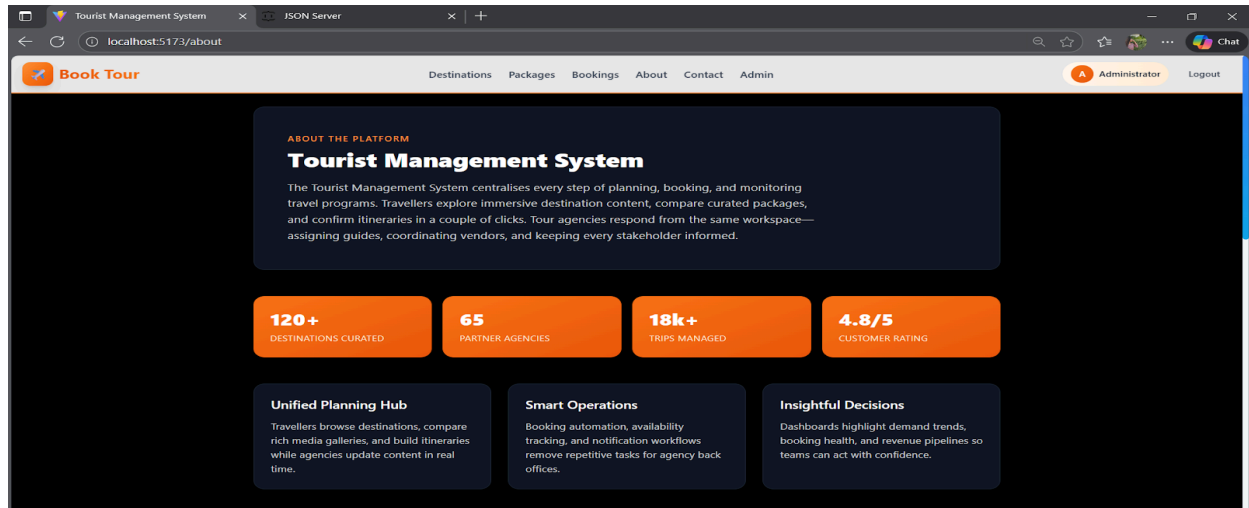
Home Page

Displays featured destinations and important sections for quick navigation



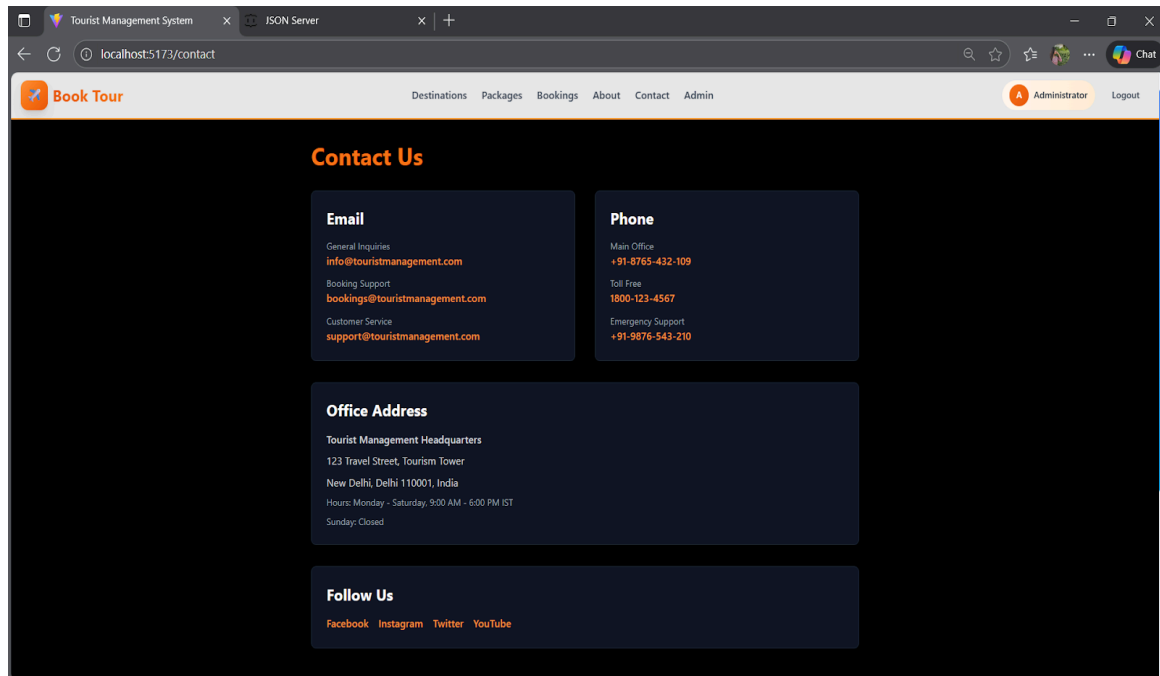
About Page

Describes the purpose and functionality of the platform.



Contact Page

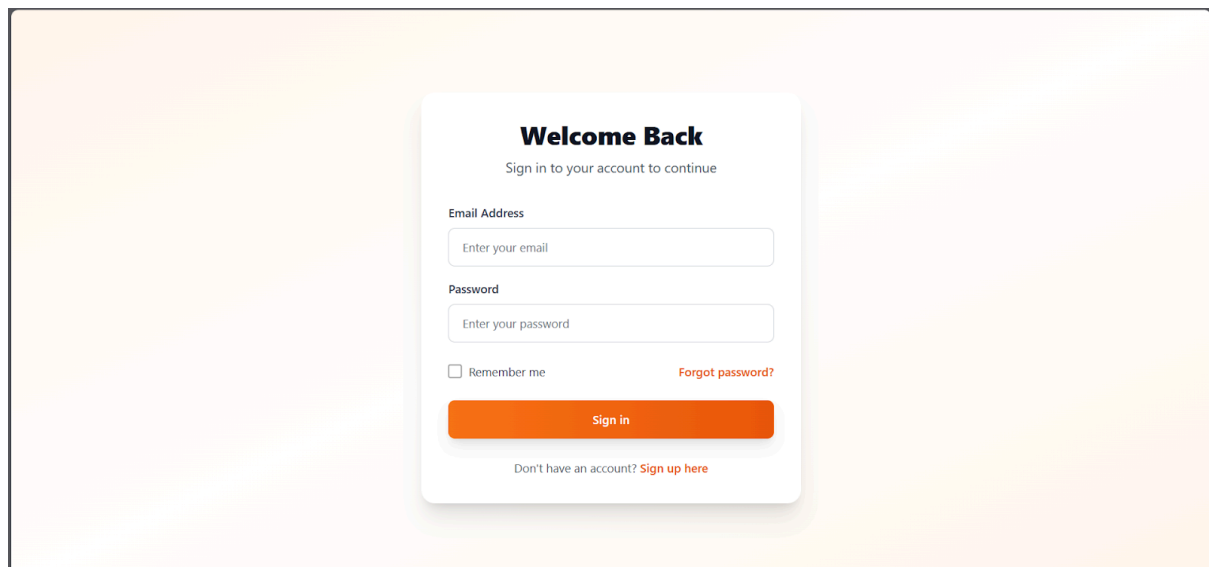
Allows users to connect with the service for further help.



Login & Register Pages

Contain authentication forms with frontend validation.

Successful login generates a secure token for protected navigation.



SignUp Here

Join us and start your journey

Full Name

Enter your full name

Email Address

Enter your email

Password

Create a password (min. 6 characters)

Confirm Password

Confirm your password

☐ I agree to the [Terms and Conditions](#)

Create Account

Already have an account? [Sign in here](#)

Destinations Page

Lists all available travel places using dynamic cards.

Tourist Management SystemJSON Server

localhost:5173/packagesSummarize

Book Tour

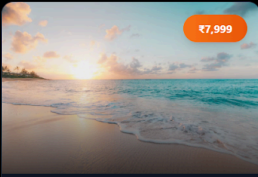
DestinationsPackagesBookingsAboutContactAdmin

AdministratorLogout

CURATED EXPERIENCES

Handpicked journeys for every traveller

Mix and match accommodations, experiences, and transfers in minutes. Choose from our expertly curated packages designed to create unforgettable memories.



₹7,999

Goa Weekend Getaway
3 days, 2 nights – Beach paradise with water sports and nightlife
[View details >](#)



₹15,999

Rajasthan Heritage Tour
5 days, 4 nights – Explore magnificent forts and desert magic
[View details >](#)

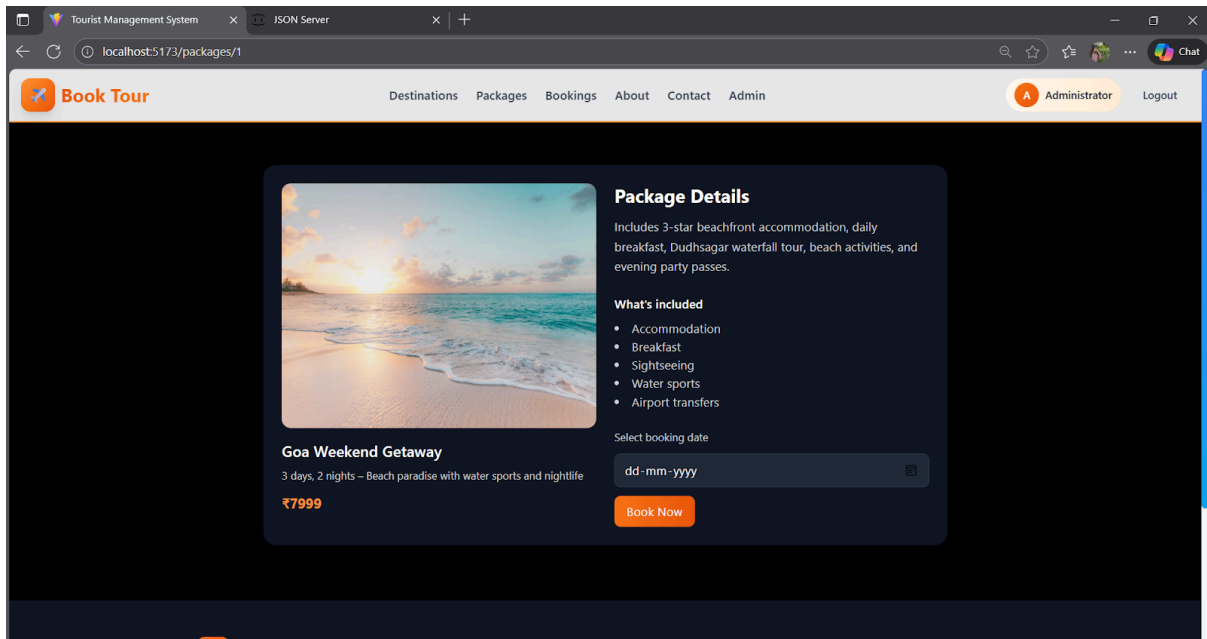


₹12,999

Kerala Backwater Cruise
4 days, 3 nights – Houseboat adventure through tropical paradise
[View details >](#)

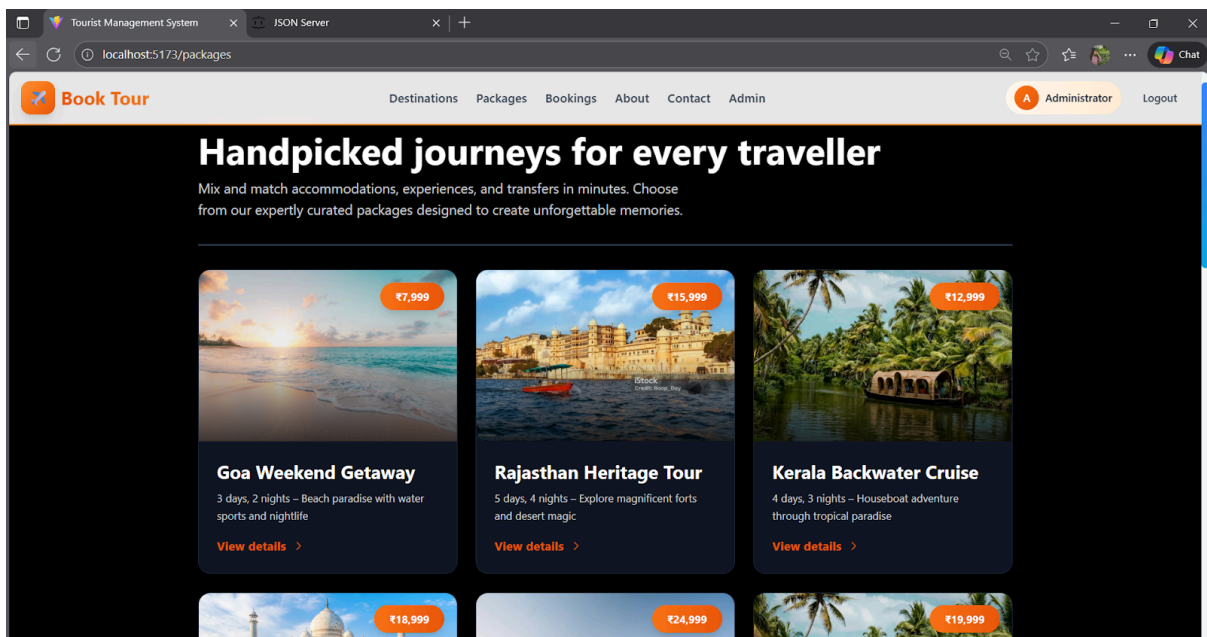
Destination Details Page

A detailed view of a selected destination with images, descriptions, and pricing.

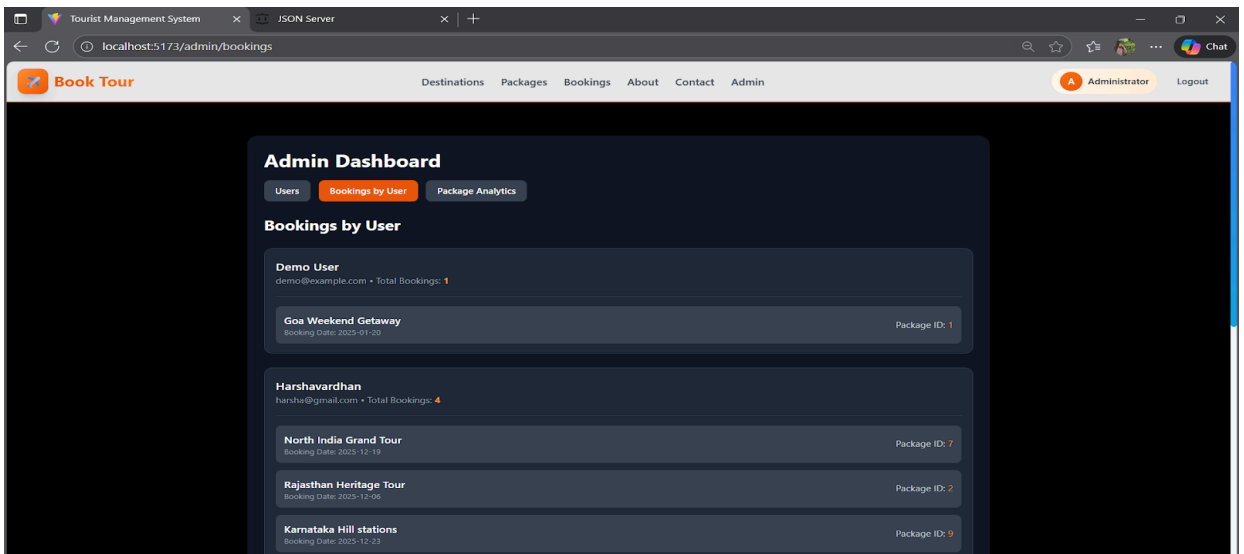
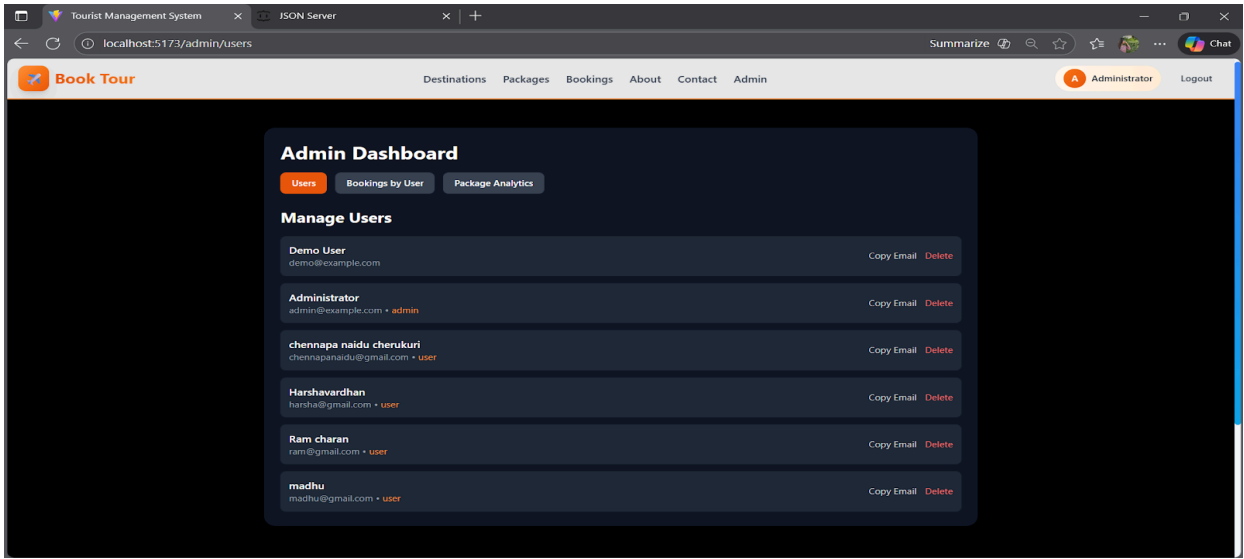


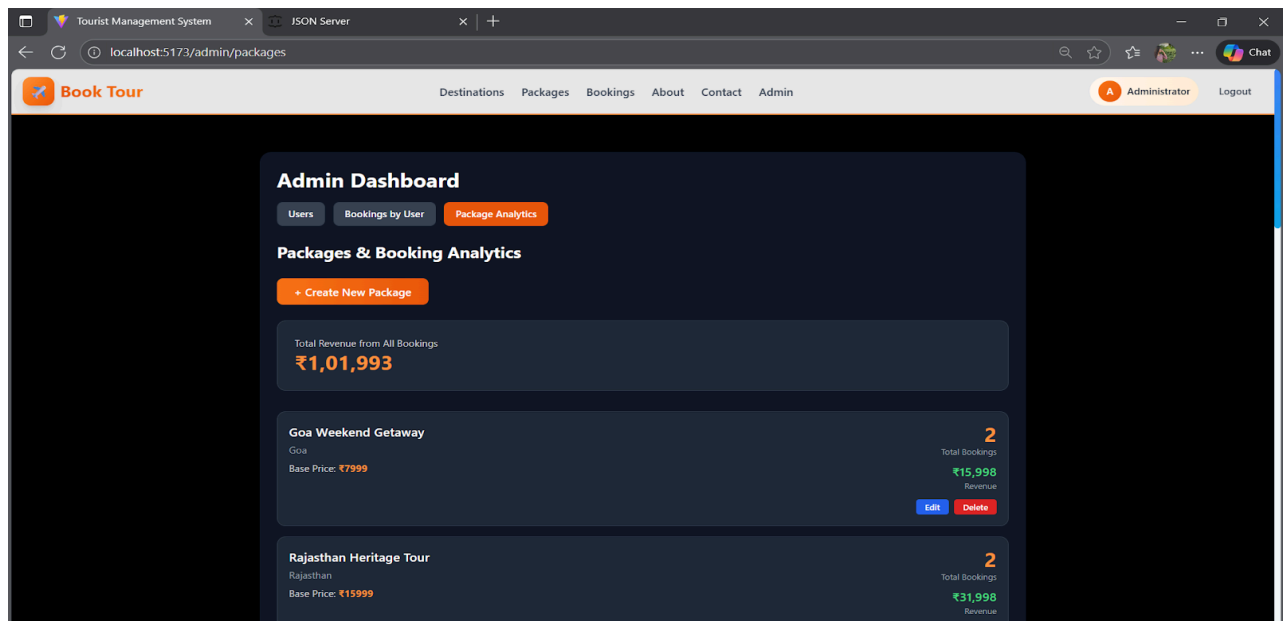
Packages Page

Shows available travel packages with key highlights.



Admin Page :





Future Enhancements

To improve the scalability, usability, and overall user experience of the Travel Booking Web Application, the following future enhancements can be implemented:

1. Online Payment Integration

Add payment gateways such as Razorpay, Stripe, or PayPal to allow users to complete bookings with secure online payments.

2. Advanced Search and Filtering

Enable users to filter destinations and packages by price, location, duration, season, ratings, etc., for a more customized browsing experience.

3. User Profile & Booking History

Allow users to manage their profiles, update personal details, and view their previous booking history.

4. Wishlist / Save for Later

Users can save destinations or packages they are interested in and view them later.

5. Reviews and Ratings

Enable users to provide feedback, ratings, and comments for destinations and packages, improving transparency and helping new customers decide.

6. Admin Analytics Dashboard

Provide admins with visual analytics such as charts for:

- Most booked destinations
- Monthly booking reports
- User growth statistics

7. Email & SMS Notifications

Send automated booking confirmation, cancellation, or reminder messages to users.

8. Multi-language Support

Include different language options so users from various regions can use the application easily.

9. Responsive Mobile App Version

Develop a cross-platform mobile application using React Native to extend accessibility beyond the web browser.

10. AI-based Recommendation System

Recommend destinations or packages based on user preferences, browsing history, and trending travel packages.

11. Chatbot Integration

Integrate a chatbot for answering user queries in real-time, improving customer support.

12. Multi-role User System

Add more roles such as:

- Travel Agent
- Manager
- Customer Support

Each with different permissions.

Extended Key Features

- Responsive design ensures compatibility across laptops, tablets, and mobile phones
- Dynamic routing for seamless navigation
- Centralized component-based architecture
- Secure authentication with protected admin routes
- Detailed views for destinations and packages
- Integrated CRUD operations for booking management
- Clean visual design using Tailwind CSS spacing, colors, and layouts
- Organized folder structure for easy scalability

Conclusion

The Travel Booking Web Application provides a streamlined and engaging solution for exploring travel destinations and managing bookings.

It eliminates complexities associated with traditional travel booking by offering a fully digital, responsive, and intuitive interface.

The project demonstrates strong frontend development skills, including routing, state handling, UI design, and secure API integration.

The architecture is scalable and adaptable for future enhancements, such as:

- Adding advanced search & filtering
- Integrating a payment gateway
- Allowing user profiles and booking history
- Adding reviews and ratings
- Admin analytics (charts, dashboards)

The project fulfills its objectives effectively and serves as a practical example of building a real-world web application using React.js.